

Thought for 124 seconds

Okay, so I need to write a Python function to find all prime numbers below 100 using the most efficient algorithm I know. Hmm, primes are numbers greater than 1 that have no divisors other than 1 and themselves. The straightforward way is to check each number up to 100, but that might not be efficient. Let me think about efficient algorithms.

Oh right, the Sieve of Eratosthenes is a classic and efficient method for finding primes up to a certain limit. It works by iteratively marking the multiples of each prime starting from 2. The steps are something like: create a list of all numbers up to n , start with the first prime (2), mark all multiples of 2 as not prime, then move to the next unmarked number (3), mark its multiples, and so on until you've processed all numbers up to $\sqrt{n}$. Since 100 is a square number, $\sqrt{100}$ is 10, so we only need to check primes up to 10. That sounds efficient because it eliminates multiples in bulk rather than checking each number individually.

Wait, how does the sieve exactly work? Let me recall. You initialize a boolean list where all entries are initially True (assuming they are primes), then set the entries to False for 0 and 1 (since they aren't primes). Then for each number starting from 2, if it's still marked as prime (True), you mark all of its multiples as not prime (False). The sieve works by eliminating non-primes in this way.

So for numbers up to 100, we can create a list of 101 elements (indices 0 to 100), set all to True, then set 0 and 1 to False. Then starting from 2, iterate up to $\sqrt{100}$ which is 10. For each prime number (where the list is still True), mark all multiples starting from p^2 up to 100, incrementing by p each time. Wait, or do we start from $2p$? *Let me think. For example, when p is 2, we start marking from 4, 6, 8, etc. So yes, starting from p^2 .* But wait, I remember that in the optimized sieve, you can start from p squared because smaller multiples would have been already marked by smaller primes. Wait, no. For example, when p is 3, the multiples 6 (32) *would have been marked by 2 already. So starting from p squared (9) would avoid redundant marking. That's a possible optimization. So for each prime p , start marking from pp up to n , stepping by p .* That way, you don't re-mark numbers already marked by smaller primes. But does that cover all multiples? Let's see: for $p=2$, the multiples are 4,6,8,... So starting from $2*2=4$. For $p=3$, start from 9, 12, 15,... but wait, 6 was already marked by 2. So yes, starting from p squared reduces redundant operations. But how does this affect the algorithm's steps?

So the sieve algorithm steps would be:

1. Create a list of booleans of size $n+1$ (for numbers 0 to n), initialized to True except indices 0 and 1 which are False.
2. For each number p starting from 2 up to $\sqrt{n}$:
 - a. If the entry at p is True, then it's a prime.
 - b. Mark all multiples of p starting from $p*p$ up to n as False, stepping by p .
3. After processing all p up to $\sqrt{n}$, the remaining True entries are primes.

Wait, but why up to $\sqrt{n}$? Because any composite number greater than $\sqrt{n}$ must have a factor less than or equal to $\sqrt{n}$, so they would have been marked already by those factors. So once we've checked up to $\sqrt{n}$, all remaining unmarked numbers are primes.

But in the case of $n=100$, $\sqrt{n}=10$. So we check primes up to 10. So for p in 2 to 10, if p is prime, mark multiples from $p*p$ onwards.

But how do we iterate p ? Because p can be 2,3,5,7 (primes up to 10). But how do we loop through p ? Since initially, the sieve starts with all True, the loop would check p from 2 to 10. For each p in that range, if $\text{sieve}[p]$ is True, then we proceed to mark multiples.

So the code outline in Python would be:

```
def primes_below_100():
    sieve = [True] * 101 # 0 to 100
    sieve[0] = sieve[1] = False
    for p in range(2, int(100**0.5) + 1):
        if sieve[p]:
            for multiple in range(p*p, 101, p):
                sieve[multiple] = False
    primes = [i for i, is_prime in enumerate(sieve) if is_prime]
    return primes
```

Wait, but let's check the range for p . The upper limit in the first for loop is $\sqrt{100}$ which is 10. So the range should be from 2 to 10 inclusive? Because in Python, range is exclusive on the upper end. So $\text{range}(2, 11)$ would include numbers 2-10. So in code, the loop is for p in $\text{range}(2, \text{int}(100^{0.5}) + 1)$. **Since 10 is $\sqrt{100}$, so the code would be $\text{int}(100^{0.5})$ which is 10, so the range is 2 to 10+1? Wait, no: $100^{0.5}$ is 10.0, so $\text{int}(10.0)$ is 10. Then $\text{range}(2, 10)$ would be up to 9. Wait, so the upper limit in range is exclusive. So to include 10, we need to do $\text{range}(2, \text{int}(10) + 1) \rightarrow 11$. So $\text{range}(2, 11)$ would generate 2-10. So yes, the code should be for p in $\text{range}(2, \text{int}(100^{0.5}) + 1)$. But 100 is fixed here. So that part is correct.**

Wait, but in the sieve algorithm, even if you don't stop at $\sqrt{n}$, but go all the way up to n , it would still work. But it's redundant because once you pass $\sqrt{n}$, there are no new multiples to mark. For example, if you have $p=11$, but 11 squared is 121, which is beyond 100. So in this case, since $n=100$, when p reaches 11, $p*p$ is 121 which is beyond 100, so the inner loop won't run. Thus, even if you loop p all the way to 100, the sieve would still be correct, but it's more efficient to stop at $\sqrt{n}$.

So in the code, for the sieve up to 100, the loop for p goes from 2 to 10.

Once that's done, the primes are the indices where $\text{sieve}[i]$ is True.

Testing this code: Let's see. For $p=2$: $\text{sieve}[2]$ is True. So mark all multiples starting from 4 ($2*2$) up to 100, step 2. So 4,6,8,...100 are marked False.

Then $p=3$: $\text{sieve}[3]$ is still True. Mark multiples starting from 9 ($3*3$), step 3: 9,12,15,...99. But 6 was already marked by 2.

Then $p=4$: $\text{sieve}[4]$ is now False (marked by 2). So skip.

$p=5$: $\text{sieve}[5]$ is True. Mark starting from 25 ($5*5$), step 5: 25,30,...100.

p=6: sieve[6] is False.

p=7: sieve[7] is True. Start from 49 ($7*7$), step 7: 49,56,63,70,77,84,91,98.

p=8: sieve[8] is False.

p=9: sieve[9] is False (marked by 3).

p=10: sieve[10] is False (marked by 2).

So after processing p=2 to 10, all non-primes up to 100 are marked. The remaining True entries are primes.

Then the primes are collected by enumerating sieve and collecting indices where the value is True.

So this should correctly return all primes below 100.

Wait, the sieve includes 100? The problem says primes below 100, so 100 is excluded. Since the sieve is up to 100, when collecting primes, we should take primes less than 100. So in the code, the sieve has 101 elements (0-100), but the primes are [i for i, is_prime in enumerate(sieve) if is_prime and i < 100]. Wait, but in the sieve creation, 100 is included. So if 100 is a prime, it would be in the list. But 100 is not a prime. So the sieve[100] would have been marked as False because 100 is even. So in the code, the primes list would automatically exclude 100. Because sieve[100] is False, since 100 is a multiple of 2.

Therefore, the code as written would return primes up to and including 100 (if any), but since the sieve marks 100 as non-prime, the primes list will be up to 97, which is the largest prime below 100. So the code is correct.

So putting it all together, the function would be as above.

But wait, in Python, when you create sieve = [True] * 101, that's 101 elements (indices 0 to 100). Then sieve[0] and sieve[1] are set to False.

Then for p in range(2, 11) (since $\text{int}(100^{*}0.5)$ is 10, so 10 +1 is 11, so the range is 2 to 11, which is 2-10 inclusive).

Then for each p in that range, if sieve[p] is True, mark multiples from p*p to 100.

Then the primes are [i for i, is_prime in enumerate(sieve) if is_prime], which includes 2-97 (primes below 100). So this function should correctly return all primes below 100.

Testing the code, let's see the primes below 100 are:

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

So the code should generate that list.

Another thing to note is that the sieve is very efficient for small numbers like 100. The sieve of Eratosthenes has a time complexity of $O(n \log \log n)$, which is more efficient than checking each number for primality individually, which would be $O(n \sqrt{n})$ or something similar.

Therefore, using the sieve is the most efficient method here.

So the Python function would be:

```
def primes_below_100():  
    sieve = [True] * 101  
  
    sieve[0] = sieve[1] = False  
  
    for p in range(2, int(100**0.5) + 1):  
  
        if sieve[p]:  
  
            for multiple in range(p*p, 101, p):  
  
                sieve[multiple] = False  
  
    return [i for i, is_prime in enumerate(sieve) if is_prime]
```

Wait, but in the code, the upper limit for the multiples is 101, but the sieve is of size 101 (indices 0-100). So the range(pp, 101, p) would go up to 100. For example, if $p=2$, $pp=4$, step 2: 4,6,...,100. So 100 is included. Which is correct because 100 is part of the sieve. Then when building the primes list, since sieve[100] is False (as it's even), it's excluded.

Thus, the code should work.

I think that's correct. Let me test some edge cases. For example, when $p=7$, the multiples start at 49. Then 49,56,63,70,77,84,91,98. All those would be marked as non-prime.

What about 11? Since the loop for p stops at 10, 11 is not checked. But sieve[11] is True, so it remains True, and since 11 is a prime, it's included in the primes list.

Yes. So this should work.

To find all prime numbers below 100 efficiently, the Sieve of Eratosthenes algorithm is an optimal choice. This algorithm works by iteratively marking the multiples of each prime starting from 2, thus eliminating non-prime numbers in a systematic way.

Approach

1. **Initialization:** Create a boolean list (sieve) of size 101 (to include numbers from 0 to 100) initialized to True, indicating that all numbers are initially considered prime.
2. **Base Cases:** Mark 0 and 1 as False since they are not primes.
3. **Marking Non-Primes:** For each number starting from 2 up to the square root of 100 (which is 10), check if it is still marked as a prime. If it is, mark all its multiples starting from its square (to avoid redundant checks) as non-prime.
4. **Collect Results:** After processing, collect all indices that are still marked True in the sieve, which represent the prime numbers below 100.

Solution Code

```
def primes_below_100():  
    sieve = [True] * 101  
    sieve[0], sieve[1] = False, False
```

```
for p in range(2, int(100 ** 0.5) + 1):
    if sieve[p]:
        for multiple in range(p * p, 101, p):
            sieve[multiple] = False
return [i for i, is_prime in enumerate(sieve) if is_prime]
```

Explanation

1. **Sieve Initialization:** The sieve list is initialized to `True` for all elements, then 0 and 1 are set to `False` because they are not primes.
2. **Prime Marking Loop:** The outer loop runs from 2 to 10 (inclusive). For each number `p` in this range, if `sieve[p]` is `True`, it means `p` is a prime. We then mark all multiples of `p` starting from `p*p` up to 100 as `False`.
3. **Efficiency Note:** By starting from `p*p` and incrementing by `p`, we avoid re-marking multiples that have already been marked by smaller primes, which improves efficiency.
4. **Result Compilation:** Finally, the indices of the sieve that are still `True` are collected into a list, which contains all prime numbers below 100.

This approach ensures that each composite number is marked exactly once by its smallest prime factor, leading to an efficient $O(n \log \log n)$ time complexity, making it the most efficient method for finding primes up to a given limit.